



Nombre:

Grupo:

Pregunta 1 (2 puntos):

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

- [] Si la clase Class A tiene el siguiente el siguiente atributo: `map<string, class B> *atributo;` Entonces necesariamente la relación entre Class A y Class B es de asociación.

[] En tablas de dispersión con cubetas, si al buscar llegamos a un hueco no siempre se detiene la búsqueda.

[] Un heap no puede estar balanceado si los valores se insertan en orden contrario al que se desea recuperar.

[] En una lista simplemente enlazada el borrado del último elemento requiere tiempo lineal incluso si se mantiene un puntero al mismo.

[] Al igual que los índices simples, los árboles B son índices en parte mantenidos en memoria principal. Su ventaja respecto a los índices es la mayor eficiencia de búsqueda: $O(\log_2 n)$ de los índices simples frente a $O(\log_m n)$ de los árboles B, siendo m el orden del mismo y siempre mayor que dos.
- [] La intersección de dos conjuntos implementados mediante vectores requiere tiempo $O(n^2)$.

[] Un vector de listas es un contenedor válido en términos de eficiencia para implementar una cola con prioridad y un grafo, aunque no tanto para una matriz dispersa.

[] En el recorrido en postorden de un ABB el orden de visita es: nodo raíz, nodo izquierda recursivo y nodo derecha recursivo.

[] Una matriz de adyacencia es una estructura adecuada para almacenar la conectividad de un grafo. Si el grafo es dirigido entonces la matriz es simétrica.

[] Las mallas regulares pueden ser muy eficientes con tiempo cercano al $O(1)$ independientemente de la distribución de los datos.

Pregunta 2 (2 puntos):

Disponemos de un fichero de registros variables (palabras en inglés) separados por el caracter “-”. El fichero dispone de una pila de registros borrados, de forma que la dirección del primer registro borrado (tope pila) está codificada en las primeras dos posiciones del fichero. Aparte el primer carácter de un registro borrado es \$ y los dos siguientes caracteres contienen la dirección del primer carácter del siguiente registro borrado o %% si se trata del último (último dato de la pila). En el ejemplo, el 12 indica que en la posición 12 comenzando por 0, está el primer registro borrado, el 29 será la posición del siguiente y así sucesivamente, El criterio para reutilizar posiciones es que la nueva palabra quepa en el hueco, sino debe ir al final. Este fichero no permite palabras con longitud inferior a 3.

12ZIP-BEING-\$29-NOW-PINK-LET-\$\$\$E-FLY-DONE

Mostrar la evolución del fichero durante las siguientes operaciones:

B(2), B(16), I(YES), I(THRONE), I(YOU)

Pregunta 3 (3 puntos):

- (a) Implementar la función `vector<T> ABB<T>::InordenNR();` capaz de devolver los datos de un árbol ABB en inorden de forma no recursiva.
- (b) Implementar la función `void ListaCircularD<T>::insertar(T& dato);` que inserta un dato en una lista circular doblemente enlazada en la posición apuntada por el único puntero cola.

Pregunta 4 (3 puntos):

La UJA se ha convencido de que la gestión de accesos a las diferentes dependencias de los edificios no funciona correctamente y se ha propuesto hacer una remodelación del edificio A3 usando puertas electrónicas accionadas a través de tarjetas con un chip. De este modo, cada usuario del edificio maneja una única tarjeta que abre tantas puertas como se le asignen sin necesidad de tener llaves, evitando la problemática asociada a la pérdida de llaves o las copias no autorizadas de éstas. El siguiente diseño UML muestra el comportamiento de la clase:

Usuario: un usuario del edificio tiene un *nif*, un *nombre*, *sexo* que es importante para abrir las puertas de los baños y un *labor* o rol que desempeña en el edificio. Todos los usuarios abren las puertas de acceso o *Entrada* al edificio. Los encargados de mantenimiento abren todo el resto de puertas. Los profesores abren las salas de juntas, el baño correspondiente y su propio despacho. Los becarios sólo tienen acceso a los baños y a su laboratorio.



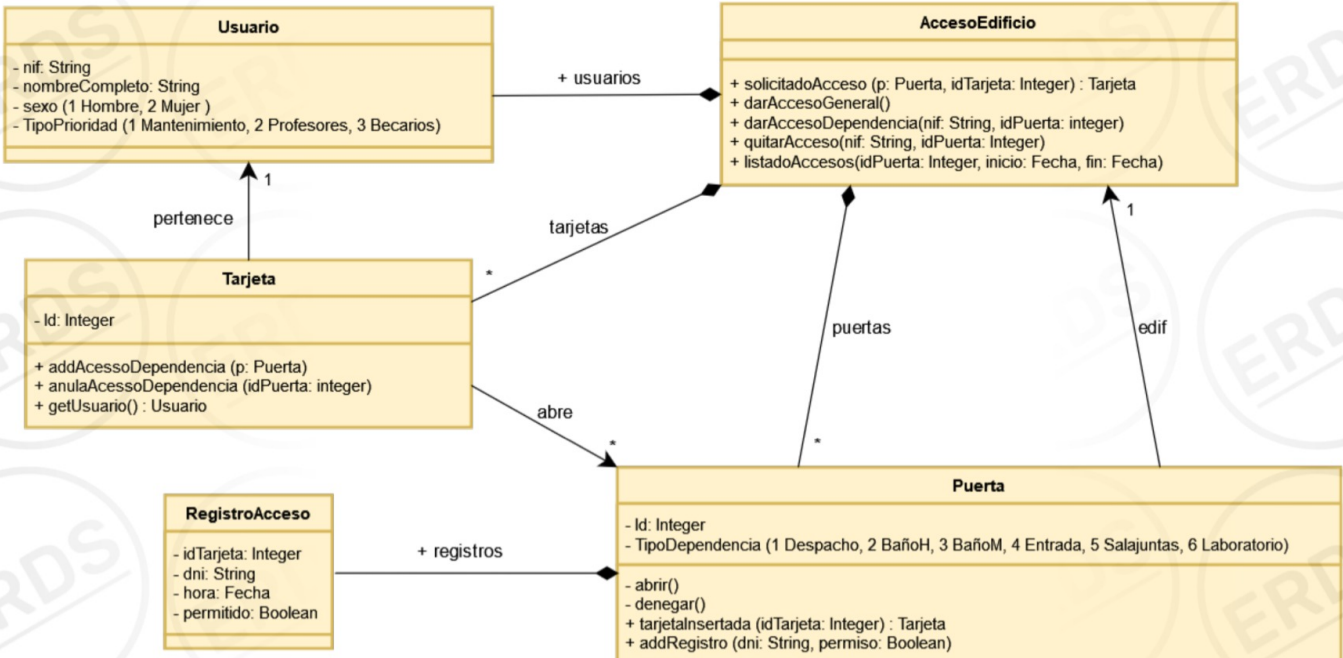
Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

Tarjeta: tiene un *Id* y hay una por usuario. Abre tantas puertas como se le asignen de forma manual (despachos y laboratorios) o automática (baños, etc.) según la descripción anterior Permiten añadir accesos con `addAccesoDependencia()` o anularlos con `anularAccesoDependencia()`.

Puerta: da acceso a una dependencia. Se le asigna un *Id* y un *TipoDependencia* según se indica en el esquema tiene un mecanismo automatizado que se activa cuando alguien mete una tarjeta con `TarjetaInsertada()`. En ese momento, lee el identificador de la tarjeta y abre o no la puerta si esa tarjeta tiene permiso para abrir dicha puerta. Para saberlo se llama a `AccesoEdificio::solicitadoAcceso()`, que se encarga de buscar si existe la relación de tarjeta con dicha puerta. Si es así, devuelve dicha tarjeta, y en caso contrario devuelve nulo. Si se concede el acceso, ya se puede llamar a `abrir()`, función que no implementa la parte mecánica pero que sí añade un registro de acceso, `RegistroAcceso` a a lista de registros con `addRegistro()`. Se deniega el acceso con `denegar()`, que también guarda un registro como se indica a continuación.

RegistroAcceso: Es una clase que sólo sirve para listar los accesos permitidos y no permitidos (permitido=True/False). Tanto si se permite o no el acceso, se registra el Id de la tarjeta insertada, el dni de la persona propietaria, la hora del sistema (usar como: `std::time_t result = std::time(nullptr); std::cout << std::asctime(std::localtime(&result));`) y un valor lógico indicando si se ha permitido o no el acceso.

AccesoEdificio: gestiona puertas, tarjetas y usuarios del edificio. El constructor de la clase toma los nombres de ficheros de usuarios, puertas y tarjetas (inventar los nombres para que entren como parámetro), añadiendo todos los usuarios, puertas y tarjetas con funciones de lectura de ficheros ya supuestamente implementadas (leerUsuarios, leerPuertas, leerTarjetas ya implementadas). Lo que no hace el constructor es asociar puertas a las tarjetas Estas operaciones se hacen con: `darAccesoGeneral()` donde se da acceso a todos los usuarios de forma genérica es decir: se da acceso total a los de mantenimiento, a los profesores a los baños y salas de juntas y a los becarios a los baños según se ha especificado para cada usuario. De forma individual se darán permisos a despachos y laboratorios con `darAccesoDependencia()`. La operación `solicitadoAcceso()` es llamada por Puerta como se ha explicado anteriormente. Finalmente `listadoAccesos()` da un listado por pantalla de todos los datos de todos los accesos fallidos y válidos de una puerta en un intervalo de tiempo entre dos fechas. Asumimos que la clase `time_t` realiza las operaciones de comparación necesarias.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional